

第四章 分支限界法

Chapter 4 Branch and Bound

计算机学院 高翔
gaoxg@nwpu.edu.cn

目 录

4.1分支限界法基本思想

4.2 电子老鼠闯迷宫

4.3 跳马问题

4.4 独轮车问题

4.5 分油问题

4.6 八数码问题

4.7 单源最短路径问题

4.1 分支限界法的基本思想

1. 分支限界法与回溯法的不同

（1）求解目标：回溯法的求解目标是找出解空间树中满足约束条件的所有解，而分支限界法的求解目标则是找出满足约束条件的一个解，或是在满足约束条件的解中找出在某种意义下的最优解。

（2）搜索方式的不同：回溯法以深度优先的方式搜索解空间树，而分支限界法则以广度优先或以最小耗费优先的方式搜索解空间树。

2. 分支限界法基本思想

分支限界法常以广度优先或以最小耗费（最大效益）优先的方式搜索问题的解空间树。

在分支限界法中，每一个活结点只有一次机会成为扩展结点。活结点一旦成为扩展结点，就一次性产生其所有儿子结点。在这些儿子结点中，导致不可行解或导致非最优解的儿子结点被舍弃，其余儿子结点被加入活结点表中。

此后，从活结点表中取下一结点成为当前扩展结点，并重复上述结点扩展过程。这个过程一直持续到找到所需的解或活结点表为空时为止。

3. 常见的两种分支限界法

（1）队列式(FIFO)分支限界法

按照队列先进先出（FIFO）原则选取下一个节点为扩展节点。

（2）优先队列式分支限界法

按照优先队列中规定的优先级选取优先级最高的节点成为当前扩展节点。

优先级队列

在许多应用中需要另一种队列，每次从队列中取出的应是具有最高优先权的元素，这种队列就是优先级队列。

在相同优先级的情况下则遵照先来先服务的原则。

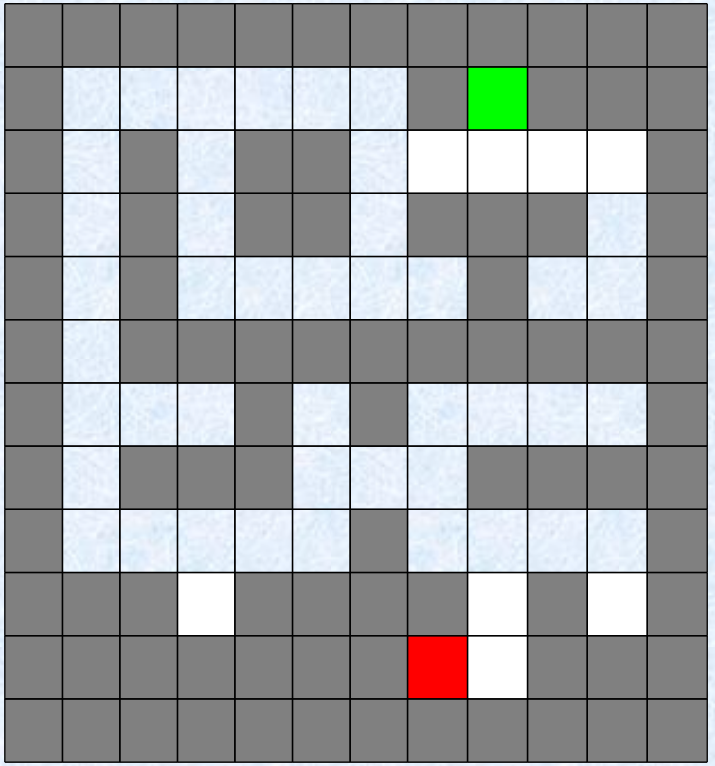
```
void search()
{
  open表初始化为空;
  起点加入到open表;
  while( open表非空 )
  {
        取open表中的一个结点u;
        从open表中删除u;
        for( 对扩展结点u得到的每个新结点vi )
        {
              if(vi是目标结点)
                    输出结果并返回;
              If(notused(vi))
                    vi进入open表;
        }
  }
}
```

```
void search ( )
{
  open表初始化为空
  起点加入open表;
  while (open表非空)
  {
    取open表中的一个结点u;
    从open表中删除u;
    for(对每
      if(vi是
```

4.2 电子老鼠闯迷宫

电子老鼠闯迷宫

如下图12×12方格图，找出一条自入口（1，8）到出口（10，7）的最短路径。



```
#include<stdio.h>
void readdata();  //读入数据
void init();      //初始化
int search(); //广搜，并在每一个可到达的每一个空格出填上最小步数
int emptyopen();  //判队列是否为空：空：1；非空：0。
int takeoutofopen(); //从队列中取出一个元素，并把该元素从栈中删除
int canmoveto(int,int,int*,int*,int);  //判能否移动到该方向，并带回坐标(r,c)
int isaim(int row, int col);        //判断该点是否是目标
int used(int,int);                  //判断该点是否已经走过
void addtoopen(int,int);            //把该点加入到open表
int a[12][12];      //a存放迷宫，0表示空格，-2表示墙。
                    //广搜时，未找到目标以前到达的空格，填上到达该点的最小步数
int n;              //n为迷宫边长，注：若大于12，必须修改一些参数，如a的大小
int open[20],head,tail,openlen=20;  //open表
int s,t;                            //起点和终点20，127
```

```
int main()
{
    int number;
    readdata();          //读取数据
    init();              //初始化
    number=search();     //广搜并返回最小步数
    printf("%d",number);  //打印结果
}
```

```
int search()
{
    int u, row, col, r, c, i, num;
    while(!emptyopen())          //当队列非空
    {
          u=takeoutofopen();     //从队列中取出一个元素，并把该元素从队列中删除
          row=u/n;               //计算该点的坐标
          col=u%n;
          num=a[row][col];       //取得该点的步数
            for(i=0;i<4;i++)
          {
             if(canmoveto(row,col,&r,&c,i)) //判能否移动到该方向，并带回坐标(r,c)
             { if(isaim(r,c))             //如果是目标结点
               return(num+1);     //返回最小步数
               a[r][c]=num+1;     //记录该点的最小步数
               addtoopen(r,c);    //把该点加入到open表
              }//end if
            }//end for
    }//end while
}//end search
```

```
int emptyopen()
{
    if(head==tail)
        return(1);
    else
        return(0);
}
```

```
int takeoutofopen()
{
   int u;
   if(head==tail)
   {
        printf("error: queue is empty"
   );
        return(-1);
   }
   u=open[head++];
   head=head%openlen;
   return(u);
}
```

```
int canmoveto(int row, int col, int *p, int *q, int direction)
{ int r, c; r=row;        c=col;
  switch(direction)
  {       case 0: c--;          //左
          break;
          case 1: r++;          //下
          break;
          case 2: c++;          //右
          break;
          case 3: r--;          //上
           break;
  }
*p=r;
*q=c;
     if(r<0||r>=n||c<0||c>=n)
//如果越界返回0
          return(0);
     if(a[r][c]==0)
//如果是空格返回1
          return(1);
     return(0);          //其余情况返回0
}
```

```
int isaim(int row, int col)
{
    if(row*n+col==t)
        return(1);
    else
        return(0);
}
```

```
int used(int row, int col)
{
    if(a[row][col]==0)      // 0表示空格
        return(0);
    else
        return(1);
}
```

```
void addtoopen(int row, int
    col)
{
    int u;
    u=row*n+col;
    open[tail++]= u;
    tail=tail%openlen;
}
```

```
void init()
{
    head=0;
    tail=1;
    open[0]=s;
}
```

4.3 跳马问题

跳马

给一个200×200的棋盘，问国际象棋的马从给定的起点到给定的终点最少需要几步。

```
#include<stdio.h>
void readdata();              //读入数据
void init();                  //初始化
int search();                 //广度优先搜索
int emptyopen();              //判队列是否为空：空：1；非空：0。
long takeoutofopen();         //从队列中取出一个元素，并把该元素从队列中删
    除
int canmoveto(int,int,int*,int*,int);   //判能否移动到该方向，并带回坐标(r,c)
int isaim(int row, int col);            //判断该点是否是目标
int used(int,int);                      //判断该点是否已经走过
void addtoopen(int,int);                //把该点加入到open表
int a[200][200],n=200;                  //a存放棋盘，n为迷宫边长
long open[2000],head,tail,openlen=2000;
long s,t;                               //起点和终点
```

```
int main()
{
    int number;
    readdata();          //读取数据
    init();              //初始化
    number=search();     //广搜并返回最小步数
    printf("%d",number);  //打印结果
}
```

```
int search()
{
    long u;
    int row, col, r, c, i, num;
    while(!emptyopen())        //当队列非空
    {
        u=takeoutofopen();
//从队列中取出一个元素，并把该元素从队列
    中删除
        row=u/n;              //计算该点所在的行
        col=u%n;              //计算该点所在的列
        num=a[row][col];      //取得该点的步
    数
for(i=0;i<8;i++)
        {
            if(canmoveto(row,col,&r,&c,i))
        //判能否移动到该方向，并带回坐标(r,c)
            {
                if(isaim(r,c))
        //如果是目标结点
                    return(num+1);
        //返回最小步数
                if(!used(r,c))
        //如果(r,c)还未到达过
                {
                    a[r][c]=num+1;
        //记录该点的最小步数
                    addtoopen(r,c);
        //把该点加入到open表
                }
            }//end if
        }//end for
    }//end while
    return -1;}
```

```
int emptyopen()
{
    if(head==tail)
        return(1);
    else
        return(0);
}
```

```
long takeoutofopen()
{
   long u;
   if(head==tail)
   {
        printf("errer: open table is empty");
        return(-1);
   }
   u=open[head++];
   head=head%openlen;
   return(u);
}
```

```
int used(int row, int col)
{
    if(a[row][col]==0)
        return(0);
    else
        return(1);
}
```

```
void addtoopen(int row, int col)
{
    int u;
    if((tail+1)%openlen==head)//有改动
        printf("open table overflow");
    u=row;
    u=u*n+col;
    open[tail++]= u;
    tail=tail%openlen;
}
```

```
void readdata()
{
    long row,col;
    scanf("%ld%ld",&row,&col);  //起点坐标
    s=row*n+col;
    scanf("%ld%ld",&row,&col);  //终点坐标
    t=row*n+col;
}
```

```
void init()
{
    head=0;
    tail=1;
    open[0]=s;
}
```

```
int isaim(int row, int col)
{
    if(row*n+col==t)
        return(1);
    else
        return(0);
}
```

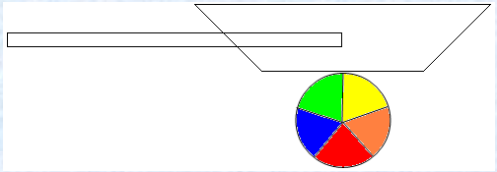
4.4 独轮车问题

独轮车

独轮车的轮子上有5种颜色，每走一格颜色变化一次，独轮车只能往前推，也可以在原地旋转，每走一格，需要一个单位的时间，每转90度需要一个单位的时间，转180度需要两个单位的时间。现给定一个20×20的迷宫、一个起点、一个终点和到达终点的颜色，问独轮车最少需要多少时间到达。

状态：独轮车所在的行、列、当前颜色、方向。

另外为了方便在结点中加上到达该点的最小步数。



```
#include<stdio.h>
struct colornode
{
   int row;                    //该状态的行
   int col;                    //       列
   int color;                   //       颜色
   int direction;                  //       方向 0，1，2，3
   int num;                    //       最小步数
};
int search();                      //广搜返回目标结点的最小步数
void readdata();                      //读入数据
void init();                     //初始化
```

```
struct colornode moveahead(struct colornode u);
//返回u向前走一格得到的结点
int used(struct colornode v);
//判断该结点是否是到达过的结点
void addtoopen(struct colornode v);  //加入到open表
int islegal(struct colornode v);
//如果该结点是合法的结点(未越界且是空格)
int isaim(struct colornode v);
//判断该结点是否是目标结点
struct colornode takeoutofopen();
//从open表中取出一个结点并把该结点从open表中删除
struct colornode turntoleft(struct colornode u);    //u向左转得到新结
  点v
struct colornode turntoright(struct colornode u);   //u向左转得到新
  结点v
```

```
struct colornode s,t;                //s:起始结点；t目标结点
struct colornode open[200];          //open表
int head,tail,openlen=200;           //open表相关数据
int direct[4][2]={{0,-1},{1,0},{0,1},{-1,0}};
//向左、下、右、上四个方向转时，行列的增加值
int a[20][20],n=20;                  //a数组表示迷宫；n为迷宫边长
int b[20][20][5][4];
//b数组表示搜索时的所有状态(0：未访问；1：已访问)
```

```
int main()
{
    int number;
    readdata();
    init();
    number=search();
    printf("%d\n",
    number);
}
```

```
int search()
//广搜返回目标结点的最小步数
{
    struct colornode u,v;
    while(head!=tail)
    {
        u=takeoutofopen();
        v=moveahead(u);
//u向前走一格得到新结点v
        if(islegal(v))
//如果该结点是合法的结点(未越界且是空格)
        { if(isaim(v))          //判是否是目标结点
            return(v.num);
            if(!used(v))   //如果是未到达过的结点
            addtoopen(v); }      //加入到open表

    v=turntoleft(u);        //u向左转得到新结点v
        if(!used(v))
        addtoopen(v);
    v=turntoright(u);       //u向右转得到新结点v
        if(!used(v))
        addtoopen(v);         }//end while
}//end search
```

```
int used(struct colornode v)
//判断该结点是否是到达过的结点
{
    if(b[v.row][v.col][v.color][v.direction]==0)
        return(0);
    else
        return(1);
}
```

```
void addtoopen(struct colornode v)
//加入到open表
{//应添加判断
    open[tail++]=v;
    tail=tail%openlen;
    b[v.row][v.col][v.color][v.direction]=1;
}
```

```
struct colornode takeoutofopen()
 //从open表中取出一个结点并把该
   结点从open表中删除
{
   struct colornode v;//后面添加判
   断
   v=open[head++];
   head=head%openlen;
   return(v);
}
```

```
void init()      //初始化
{
  int i,j,k,l;
  for(i=0;i<n;i++)    //所有状态初始化
        for(j=0;j<n;j++)
                for(k=0;k<5;k++)
                        for(l=0;l<4;l++)
                                b[i][j][k][l]=0;
  head=0;
  tail=1;
  addtoopen(s);   //把起始点加入到open表
}
```

```
void readdata()                    //读入数据
{
  char str[50];int i,j;
  for(i=0;i<n;i++)
  {
        gets(str);
        for(j=0;j<n;j++)
              if(str[j]=='.')        //读入数据'.'表示空格
                    a[i][j]=0;          //存储时  0：表示空格
              else
                    a[i][j]=1;          //       1：表示墙
  }
  scanf("%d%d%d%d",&s.row,&s.col,&s.color,&s.direction);  //读入起始结点信
  息
  scanf("%d%d%d",&t.row,&t.col,&t.color);              //读入目标结点信息
}
```

```
int isaim(struct colornode v)
//判断该结点是否是目标结点
{
if(v.row==t.row&&v.col==t.col&&v.color==t.col
   or)
        return 1;
else
        return 0;
}
```

```
int islegal(struct colornode v)
 //如果该结点是合法的结点(未越界且是空格)
{
    if(v.row<0||v.row>=n||v.col<0||v.col>=n)
     //若越界
          return 0;
    if(a[v.row][v.col]==0)
     //0：表示空格
          return 1;
    return 0;
}
```

```
struct colornode moveahead(struct colornode u)
//返回u向前走一格得到的结点
{
    struct colornode v;
    v.row=u.row+direct[u.direction][0];
    v.col=u.col+direct[u.direction][1];
    v.color=(u.color+1)%5;
    v.direction=u.direction;
    v.num=u.num+1;
    return(v);
}
```

```
struct colornode turntoleft(struct
    colornode u)
  //u向左转得到新结点v
{
    struct colornode v;
    v=u;
    v.direction=(v.direction+1)%4;
    v.num=v.num+1;
    return(v);
}
```

```
struct colornode turntoright(struct
    colornode u)
//u向右转得到新结点v
{
    struct colornode v;
    v=u;
    v.direction=(v.direction+3)%4;
    v.num=v.num+1;
    return(v);
}
```

```
struct colornode turntoright(struct
    colornode u)
//u向右转得到新结点v
{
    struct colornode v;
    v=u;
    v.direction=(v.direction+3)%4;
    v.num=v.num+1;
    return(v);
}
```

4.5 分油问题

分油问题

一个1斤的油瓶内装满了油。有两个空瓶，容量分别是7两和3两。

要求在三个瓶子间进行倒油，使一斤油平分为两个五两。

算法描述

1 建立一个队列，用来放入所有产生的新结点。其中记录每个状态三个瓶子里油量和每个结点，初试结点父节点置为-1；

2 建立一个链表，用来放入已产生的结点，用来与新状态比较，只需要比较前两个瓶子里的油量即可。

3 开始操作，看目前形成结点状态是不是最后要求的结果，是则停止搜索。

4 如果不是最后结果，则按智能算子的规则求出新状态，并判断是不是新状态，如果是新状态，则在搜索队列和查找链表中分别加入该状态形成的结点。

5 队列头结点出队，返回步骤3。

问题分析：

选择合适的数据结构表示问题状态，用队列和链表的结点来放置瓶内油量有关数据。

如

```
struct node{
 int container1;
 int container2;
 int container3;
 int father;
};
```

分析问题时可以把3个油瓶里的油量作为一个向量考虑：

问题的起始状态：(10，0，0)。 问题的目标状态：(5，5，0)。

确定智能算子：用以表示变化状态的规则。由于总共油量为10两，而10两的瓶可以装满所有的油，因此在描述变化状态的时候只需给出10两瓶和7两瓶的油量即可，3两瓶的油量即为10（10两瓶油量）—（7两瓶油量）。在比较结点是否存在时只需比较两个大瓶，因此链表中采用只存储两个大瓶油量的方式。在广度搜索的队列和最后寻找目标结点所在路径时存储全部三个瓶子的油量。

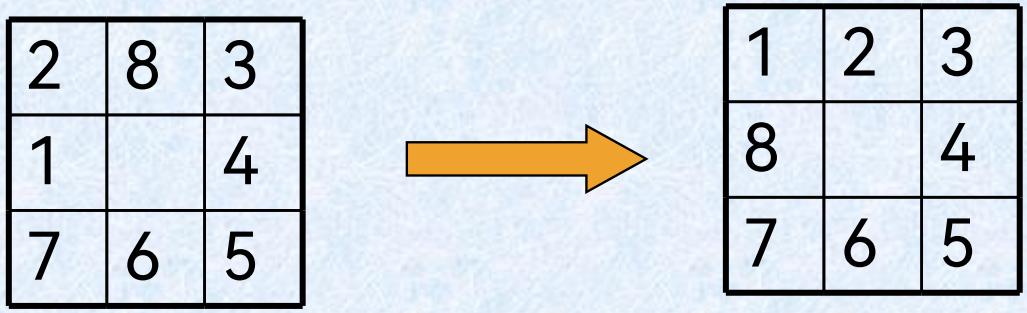
情况数	条件和操作	油量的改变
1	7两瓶中有油，全倒入10两瓶	(B1,B2,B3)→(B1+B2,0,B3)
2	3两瓶中有油，全倒入10两瓶	(B1,B2,B3)→(B1+B3,B2,0)
3	10两瓶向7两瓶倒，其中7两瓶和10两瓶中油加起来大于等于7两（小于7）	(B1,B2,B3)→(B1+B2-7,7,B3)
4	3两瓶向7两瓶倒，其中7两瓶和3两瓶中油加起来大于等于7两	(B1,B2,B3)→(B1, 7,B2+B3-7)
5	3两瓶向7两瓶倒，其中7两瓶和3两瓶中油加起来不大于7两	(B1,B2,B3)→(B1,B2+B3,0)
6	10两瓶向3两瓶倒，其中10两瓶和3两瓶中油加起来大于等于3两（小于3）	(B1,B2,B3)→(B1+B3-3,B2,3)
7	7两瓶向3两瓶倒，7两瓶和3两瓶中油加起来大于等于3两	(B1,B2,B3)→(B1,B2+B3-3,3)
8	7两瓶向3两瓶倒，7两瓶和3两瓶中油加起来不大于3两	(B1,B2,B3)→(B1,,0,B2+B3)

4.6 八数码问题

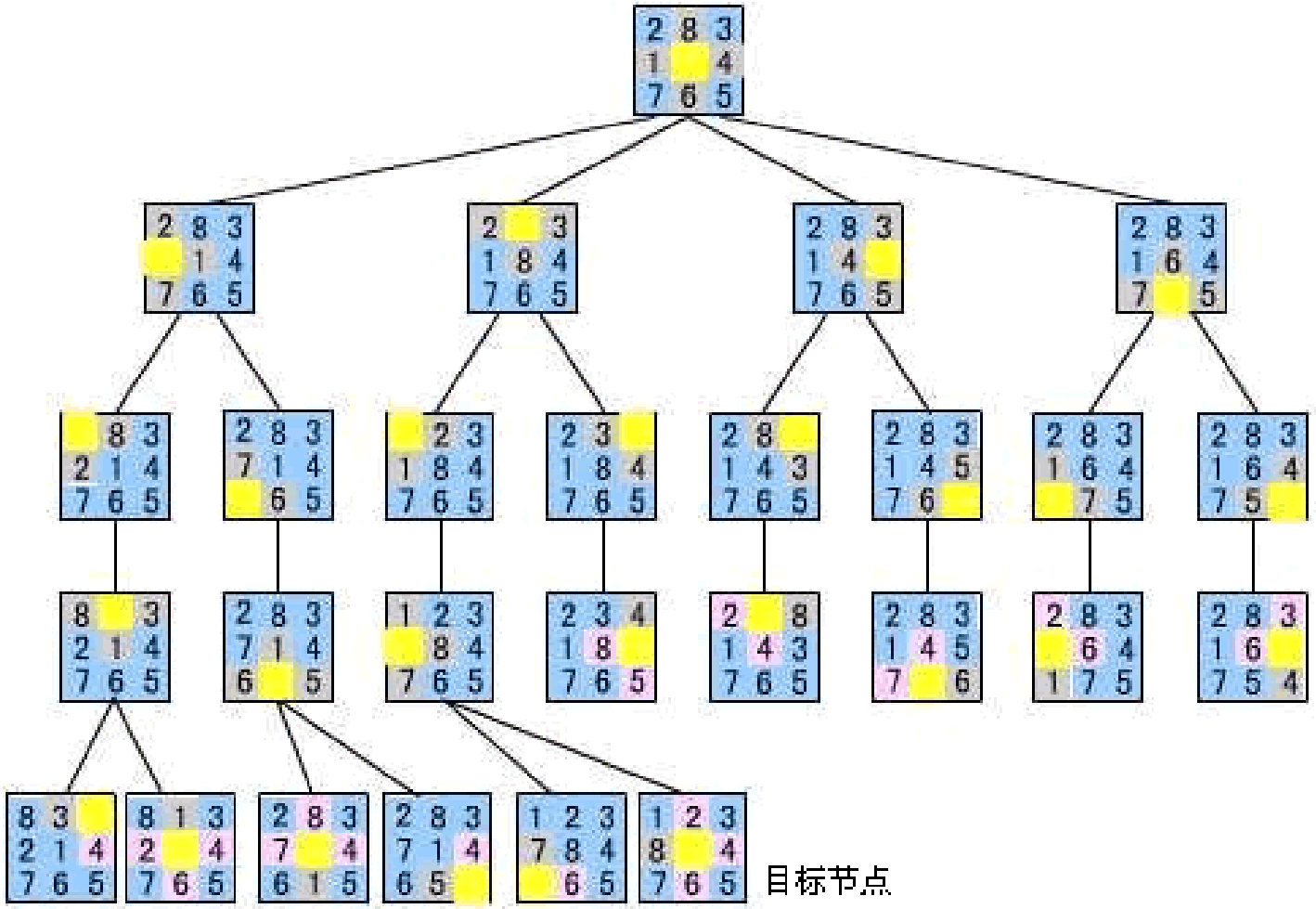
用广度优先搜索策略解八数码难题。简单的广度

由于把一个与空格相邻的数码移入空格，等价于把空格向数码方向移动一位。所以，该题中给出的数码走步规则也可以简化为：对空格可施行左移、右移、上移和下移等四种操作。

设初始节点S_0和目标节点S_g分别如图的初始棋局和目标棋局所示，我们用广度优先搜索策略，则可得到如图所示的搜索树。




2 8 3
1 4
7 6 5


目标节点

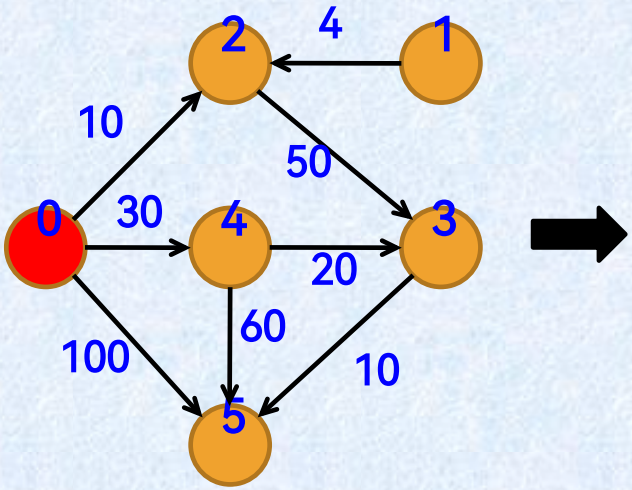
4.7 单源最短路径问题

【问题描述】给定一个带权有向图G=（V，E），其中每条边的权是一个正整数。

另外，还给定V中的一个顶点v，称为源点。计算从源点到其他所有顶点的最短路径长度。这里的长度是指路上各边权之和。

采用队列式分枝限界法求解

实例图



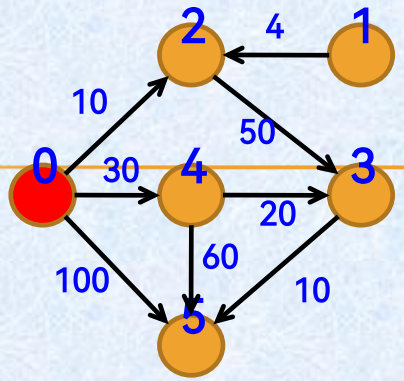
$$\begin{bmatrix} 0 & \infty & 10 & \infty & 30 & 100 \\ \infty & 0 & 4 & \infty & \infty & \infty \\ \infty & \infty & 0 & 50 & \infty & \infty \\ \infty & \infty & \infty & 0 & \infty & 10 \\ \infty & \infty & \infty & 20 & 0 & 60 \\ \infty & \infty & \infty & \infty & \infty & 0 \end{bmatrix}$$

队列结点类型声明如下：

```
struct NodeType       //队列结点类型
{  int vno;           //顶点编号
   int length;        //路径长度
};
```

用dist数组存放源点v出发的最短路径长度，dist[i]表示源点v到顶点i的最短路径长度，初始时所有dist[i]值为∞。

用prev数组存放最短路径，prev[i]表示源点v到顶点i的最短路径中顶点i的前驱顶点。



顶点编号，length

0,0 dist[*i*]=∞

0→2

0→4

0→5

0+10<∞:
prev[2]=0
dist[2]=10

2,10

0+30<∞:
prev[4]=0
dist[4]=30

4,30

0+100<∞:
prev[5]=0
dist[5]=100

5,100

2→3

4→3

4→5

10+50<∞:
prev[3]=2
dist[3]=60

3,60

30+20<60:
prev[3]=4
dist[3]=50

3,50

5,90

30+60<100:
prev[5]=4
dist[5]=90

3→5

60+10<90:
prev[5]=3

5,70

50+10<70
prev[5]=3

5,60

3→5

dist[1]=∞，prev[1]=* dist[2]=10，prev[2]=0
dist[3]=50，prev[3]=4dist[4]=30，prev[4]=0
dist[5]=60，prev[5]=3

求顶点0出发的最短路径

```
#define INF 0x3f3f3f3f  //表示∞
#define MAXN 51
//问题表示
int n;                  //图顶点个数
int a[MAXN][MAXN];      //图的邻接矩阵
int v;                  //源点

//求解结果表示
int dist[MAXN]; //dist[i]源点到顶点i的最短路径长度
int prev[MAXN];       //prev[i]表示源点到j的最短路径中顶点j的前驱顶点

struct NodeType       //队列结点类型
{  int vno;           //顶点编号
   int length;        //路径长度
};
```

```
void bfs(int v)                    //求解算法
{  NodeType e,e1;
   queue<NodeType> pqu;
   e.vno=v;                        //建立源点结点e（根结点）
   e.length=0;
   pqu.push(e);                    //源点结点e进队
   dist[v]=0;
   while(!pqu.empty())             //队列不空循环
   {  e=pqu.front(); pqu.pop();    //出队列结点e
      for (int j=0; j<n; j++)
      {  if(a[e.vno][j]<INF && e.length+a[e.vno][j]<dist[j])
         {   //剪枝：e.vno到顶点j有边并且路径长度更短
             dist[j]=e.length+a[e.vno][j];
             prev[j]=e.vno;
             e1.vno=j;       //建立相邻顶点j的结点e1
             e1.length=dist[j];
             pqu.push(e1);           //结点e1进队
         }
      }
   }
}
```

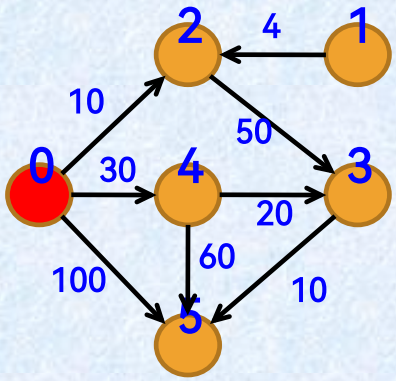
采用优先队列式分枝限界法求解

采用STL的priority_queue<NodeType>容器作为优先队列（小根堆），优先队列结点类型与前面的相同，添加比较重载函数，即按结点的length成员值越小越优先出队，为此设计NodeType结构体的比较重载函数如下：

```
bool operator<(const NodeType & node) const
{
    return length>node.length;    //length越小越优先出队
}
```

```
void bfs(int v)                    //求解算法
{  NodeType e,e1;
   priority_queue<NodeType> pqu;   //定义优先队列
   e.vno=v;                        //建立源点结点e
   e.length=0;
   pqu.push(e);                    //源点结点e进队
   dist[v]=0;

   while(!pqu.empty())             //队列不空循环
   {  e=pqu.top(); pqu.pop();              //出队列结点e
      for (int j=0; j<n; j++)
      {  if(a[e.vno][j]<INF && e.length+a[e.vno][j]<dist[j])
         {  //剪枝：e.vno到顶点j有边并且路径长度更短
            dist[j]=e.length+a[e.vno][j];
            prev[j]=e.vno;
            e1.vno=j;                //建立相邻顶点j的结点e1
            e1.length=dist[j];
            pqu.push(e1);            //结点e1进队
         }
      }
   }
}
```



顶点编号，length

0,0 dist[i]=∞

0→2

0→4

0→5

0+10<∞:
prev[2]=0
dist[2]=10

2,10

0+30<∞:
prev[4]=0
dist[4]=30

4,30

0+100<∞:
prev[5]=0
dist[5]=100

5,100

2→3

4→3

4→5

10+50<∞:
prev[3]=2
dist[3]=60

3,60

30+20<60:
prev[3]=4
dist[3]=50

3,50

5,90

30+60<100:
prev[5]=4
dist[5]=90

3→5

50+10<70
prev[5]=3

5,60

dist[1]=∞，prev[1]=* dist[2]=10，prev[2]=0
dist[3]=50，prev[3]=4dist[4]=30，prev[4]=0
dist[5]=60，prev[5]=3

求顶点0出发的最短路径